

CSC311: Introduction to Machine Learning

Project Report

The Most Naïve Implementation

Submitted by

Name	UTORid
Tajwaar Shafiq	shafiq1
David Wang	wang4743
Frederick Meneses	menese23
Syed Salman Husainie	husaini8

1 Executive Summary

In this project, we explored three model families including decision trees, neural networks, and naive bayes classifiers. Our best performing model is Random Forest with a projected accuracy of $\sim 71\%$. This model outperformed the others because it used a bagging ensemble method, reducing variance by combining predictions from multiple decision trees to capture complex non-linear patterns in the text. Random forest is generally the best black-box machine learning algorithm without necessary tuning. Neural networks are not as effective at handling outliers or high dimensional data as they are less interpretable than random forests. They also require significantly more data to generalize effectively. Because Naive Bayes is a simple probabilistic model, it does not capture the nuances a decision tree or neural network based model would account for. For classification tasks specifically, Random Forest's interpretability is important, both for performance of the task and cost, outweighing the trade-offs required to meet file size constraints.

2 Data Exploration

2.1 Data Summary

The dataset contains 10 features and one categorical label identifying the model as ChatGPT, Gemini, or Claude. The classes are well-balanced in the training data. The features include: a student identifier, five text responses, and four categorical Likert-scale ratings.

Analysis from Figure 1 reveal user perception patterns. ChatGPT is the most likely to be used for academic tasks, with 73% of students giving it a high rating (4 or 5), compared to 25% for Claude and 19% for Gemini. It is also the most verified model, with 58% of students frequently checking its responses. Conversely, it also receives the highest rate of suboptimal response reports at 36%. In contrast, Claude is perceived as providing the fewest references or supporting evidence, with 83% of students giving it a low rating on this aspect.

2.2 Data Issues

There are several empty data or corrupted data written as "#NAME?". To treat these values:

- If it is a text data, replace with the empty string "", which will be converted to a vector filled with 0.
- If it is a categorical data, fill it with a rounded average of that column in the training data.

2.3 Data Preprocessing

The output labels will be one-hot encoded for loss calculation. For the features, Likert-scale ratings will be treated as ordinal values, while the "select all" questions will be one-hot encoded. Text responses will be preprocessed by converting to lowercase, truncating words to their stems (i.e., talk, talking, talker), and replacing punctuation with spaces. A vocabulary will be built from the training set, and text will be vectorized using TF-IDF [1] and then be L2 normalized.

The dataset of 825 datapoints will be split with a priority on training data due to limited size. A small test set of 30 points ($\sim 5\%$) will be reserved for final benchmarking. The remaining data will be split 20/80 into validation (159 points) and training (636 points) sets. All splits will maintain equal class distribution.

2.4 Data Leakage Prevention

To prevent data leakage, the dataset will be split based on student IDs. This ensures that all responses from a single student are contained within either the training, validation, or test set, preventing the model from learning a student's writing style across different subsets. The splitting procedure will distribute students uniformly to maintain balanced class distributions for each model. Potential issues like temporal shifts and/or biases will not be within our consideration as the data was given to us all at once, and the model will not be tested on future datasets.

3 Methodology

3.1 Chosen Models

Decision Trees

Being a simple, yet powerful algorithm, decision trees will be able to provide us with an interpretable and explainable model for the ranked responses of our data. As the text classification of student responses will involve categorical features, the natural handling of these done by decision trees will make them an appropriate fit. Moreover, as decision trees by themselves are not affected by scaling, we do not need to conduct any normalization for features such as word frequencies.

Neural Networks

Neural networks will allow us to unlock high performance on our complex, structured, text training data. Due to their capabilities as universal function approximators, we will be able to capture complex non-linear patterns within the training data, which can be considered exceptionally important in subtle, complicated linguistic combinations in natural language text.

Naive Bayes

A Naive Bayes model will allow us to implement a simple and understandable classifier enabling generative capabilities for new data points.

Random Forests

As a relative to the Decision Tree model, Random Forests will allow us to combine results of many Decision Trees in a manner of bagging. In doing so, and by introducing some randomness, we aim to reduce variance overall within our predictions.

3.2 Optimization Techniques

Our **neural network** is trained on Sklearn's multilayer perceptron classifier. The Adam optimizer was tested to perform better in general than gradient descent, which is reasonable considering the small sample size. The sparsity of the TF-IDF text vectors could cause problems to the weights of the neural network, so in data preprocessing, the words are all truncated to the first 4 letters to reduce feature dimensions while also keeping meanings of different words. Additionally, the features are also L2 normalized to ensure effective training. In training, early stopping by monitoring validation data was also used to prevent the model from overfitting the training data. A weight regularization term was also added to prevent extreme weights which would cause high variance in predictions. Finally, Sklearn's adaptive learning rate was used, which reduces learning rate for more precise weight adjustments everytime training accuracy plateaus.

For **decision trees**, we conduct thorough hyperparameter tuning and utilizing bagging and boosting techniques (e.g. random forest, gradient boosting). Primarily, this lead to us introducing **random forests** as another model family. Sklearn's RandomForestClassifier comes built-in with optimization techniques such as increased bootstrap sampling efficiency, and various tools to balance between under and overfitting such as `max_features` to limit how many features each split considers, and those that are also present within our **decision tree** models, such as `max_depth` to limit the depth of trees to prevent overfitting.

For **naive bayes**, we use Multinoulli strategy to deal with two types of data (text and categorical). We will incorporate MLE and MAP learning. To optimize these probabilistic techniques, we maximize the log-likelihood of MLE and MAP by introducing a Laplace smoothing parameter to determine the optimal value. We will also use Beta(2, 2) for bag-of-words and Dirichlet smoothing for quantitative, categorical variables.

3.3 Validation Method

As mentioned prior, reserving around 30 datapoints for our **test set**, we divide the remaining into an **80/20** split for **training/validation** sets respectively. We will pick uniformly and evenly out of the 3 models to ensure we have an even distribution of datapoints for each model in each data set.

We will train our model using the training sets, then conduct hyperparameter tuning with the validation set, and only use our test set once afterwards, for our final model.

3.4 Evaluation Metrics

To evaluate models, we plan to use Accuracy, Recall (sensitivity), Precision, and FPR as evaluation metrics across the board, and to use confusion matrices to investigate patterns in model predictions. Accuracy is the default metric we use to evaluate the performance of model families. We use Recall and Precision to estimate the error due to text to Likert feature ratio and to measure per-class performance as accuracy weighs errors equally. We will also investigate F1 scores as the harmonic mean between precision and recall, in addition to entropy / gini for our decision tree models, alongside training and validation loss within our neural networks.

3.5 Hyperparameter Tuning

For our model families, we tune the following hyperparameters, where each corresponding range is in parentheses. We selected these ranges because they are standard and reflects the ranges seen in our labs.

- Decision Trees
 - **Max tree depth (3-100)**: We want to ensure that the trees capture complex patterns, however deeper trees run risks of overfitting
 - **Min samples split (2-400)**: This will limit the range of samples required for a split. This range will lower risks of overfitting, while allowing us to tune to capture complex patterns
 - **Criterion (entropy, gini)**: The measure of impurity used when determining splits. While gini can be more performant while generating trees, entropy can capture more nuanced splits
- Random Forests
 - **Number of estimators (50-250)**: The number of trees within the forest. We want to ensure we have a variety of random trees to reduce our variance, as well as capture complex patterns within the training set. It is also important to note here that as file size plays a key role due to the limitations set on complimentary files to `pred.py`, we are limited by what we can reasonably compress and save for all hyperpara.
 - **Max tree depth (5-30)**: We want to ensure that trees within the forest capture important patterns, whilst not overfitting to the training set.
 - ***Note:** All decision tree hyperparameters were tuned as well*
- Neural Networks
 - **Number of hidden layers and size of each:**
 - * (128)
 - * (256)
 - * (128, 64)
 - * (256, 128)

These values for hidden layers were chosen because the TF-IDF vectors resulted in around 1600 features, which means it would need decently sized hidden layers. However, the hidden layers should not be too large as it could overfit to the data.

- **L2 regularization strength (α):**
 - * 0.0001
 - * 0.001
 - * 0.01

Small values of regularization strength were tested mainly due to the large size of the TF-IDF features

compared to the numeric and one-hot features. Forcing too much weight regularization could make the model underweight the latter.

- **Initial learning rate:**

- * 0.001
- * 0.01

This is the standard range for learning rates. The initial value being too large would cause the weights to vary too much at the start, too small would make the weights take a long time to converge.

- Naive Bayes

- **Beta Prior(2, 2) for MAP:** This acts as a regularizer for the bag of words as MLE tends to perform worse on small datasets.
- **Laplace smoothing α [0, 1]:** This hyperparameter address the problem of zero probability, which occurs when a feature does not appear in the training data for a particular class. By adding a small pseudocount to all feature counts, it ensures that no probability estimates are zero, allowing for more reliable classification.
- **Dirichlet prior for MAP:** This smooths the estimates of probabilities for our quantitative categorical data.

4 Results

Table 1: Performance Comparison Across Model Families

Model	Train Acc.	Valid Acc.	Test Acc.
Decision Trees	97.5%	67.3%	66.7%
Naive Bayes (MLE)	97.3%	62.3%	53.3%
Naive Bayes (MAP)	90.1%	59.8%	60.0%
MLP (Neural Net)	84.3%	70.0%	-
Random Forest	99.5%	71.3%	74.1%

Naive Bayes Classifier MLE initially underfits with no smoothing. As seen in Figure 5, validation accuracy peaks at (62.26%) with $\alpha = 0.1$. MAP stays constant at (59.75%) due to fixed Beta(2, 2) and Dirichlet priors, default smoothing, and text to quantitative feature ratio. The drop in Validation Accuracy (62.3%) compared to Training (97.3%) indicates that the independence assumption of Naive Bayes struggles to generalize to unseen writing styles in this dataset. As we approach $\alpha = 1$, we see the MLE validation accuracy decrease and training accuracy minimized, suggesting some overfitting.

- Per-class precision for Optimal MLE on Test Data: [0.85714286, 0.58333333, 0.27272727] Per-class recall for Optimal MLE on Test Data: [0.75, 0.5, 0.375] Figure 3

Gemini is most frequently misclassified by the model suggesting that responses differ little between models, with GPT classifications corresponding to generic responses.

Neural Networks (Multi-layer Perceptron) Optimal Configuration & Performance: - Best hyperparameters: two hidden layers (256, 128), alpha=0.0001, learning rate = 0.01 - Training Accuracy: 0.8426, Validation Accuracy: 0.7000 Precision: 0.7151, Recall: 0.7000, F1-Score: 0.6823 (Figure 10)

From Figure 9, it can be seen that having two hidden layers generally performed worse than having one single hidden likely due to high complexity and more risk for overfitting. A low regularization strength and low learning rate also showed better accuracy on average. However, the values are different in the best model. This could be signs that a more complex model would perform better with detailed finetuning. The bottom two graphs show that overall, there has been not too much overfitting as both training and validation accuracy are close to each other.

We experimented with **Decision Trees** using both Bagging and Boosting, achieving approximately 70% training accuracy. This involved tuning all previously reported hyperparameters and comparing Bag-of-Words against TF-IDF vectorization. We optimize Decision Trees using **Random Forests**.

Our initial optimal forest with 500 trees achieved a 98% training accuracy and 95% validation accuracy. However, this model exceeds the 10 MB submission limit at 100 MB. We then retrained with reduced tree counts (25, 50, 75, 100), with final training accuracy 97% and validation accuracy 71%.

Optimal Configuration & Performance: **50 trees, sqrt max features, gini loss criterion, max depth 30, 2 min samples split, 1 min samples leaf**

As seen in Figure 7, and from a generated classification report, we have the following:

- Per-class precision on Validation Data: [0.80, 0.67, 0.80], Average: 0.757
- Per-class recall on Validation Data: [0.89, 0.89, 0.44], Average: 0.74
- Per-class F1 scores: [0.84, 0.76, 0.57], Average: 0.72

This suggests that similar to Naive Bayes, the Random Forest model is best at GPT classifications. Then, the model tends to have Gemini misclassifications, but does not have many false positives.

5 Contributions and Learning

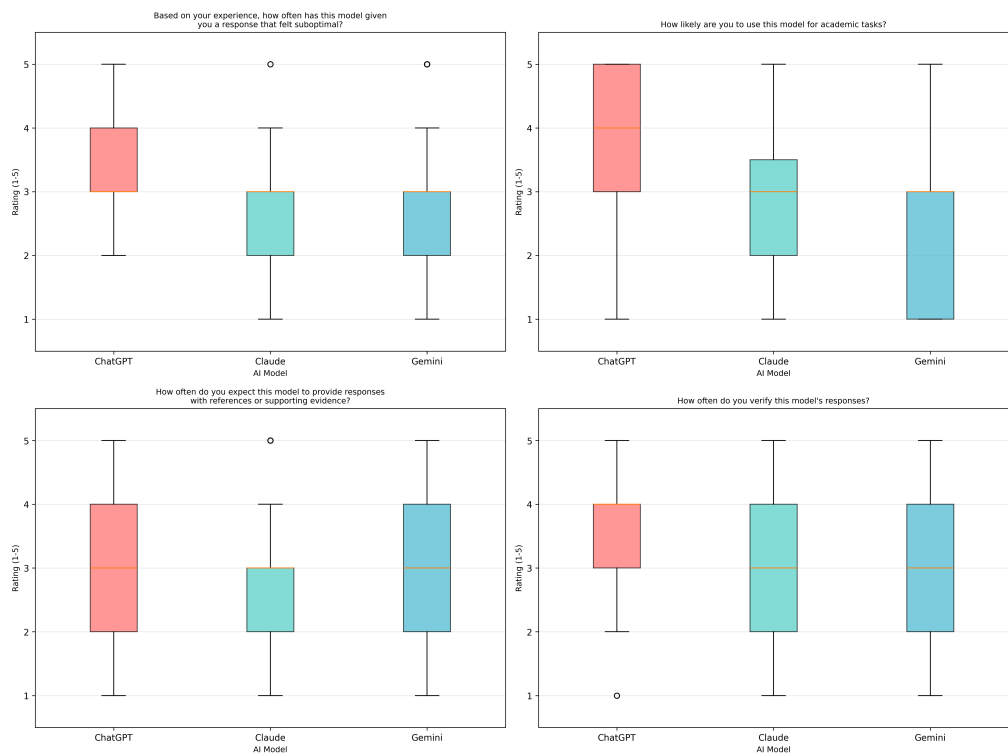
- Frederick: project lead, implementing naive bayes classifier. The most important lesson learned is don't repeat experiments over and over expecting different results.
- Salman: analyst, drafting and finalizing the project report, helping implement the naive bayes classifier, and doing code reviews. The most important lesson I learned was how vitally important data pre-processing is to make the raw data even usable by the models.
- David: tech lead, implemented data preprocessing, decision trees, and neural network models. The most important lesson I learned was the importance of having a clean and organized plan to prevent repetition of codes, such as multiple usage of dataset loading, model hypertuning, and load/saving parameters. Plenty of lines were simplified by creating helper functions and classes.
- Tajwaar: scrum master, organized deadlines and deliverables, implemented the data pre-proocessing, decision trees, and random forest models. The most important lesson learned was how real-world technical constraints (e.g. file size, memory/library limitations) can vastly impact both model accuracy, training, and performance.

6 References

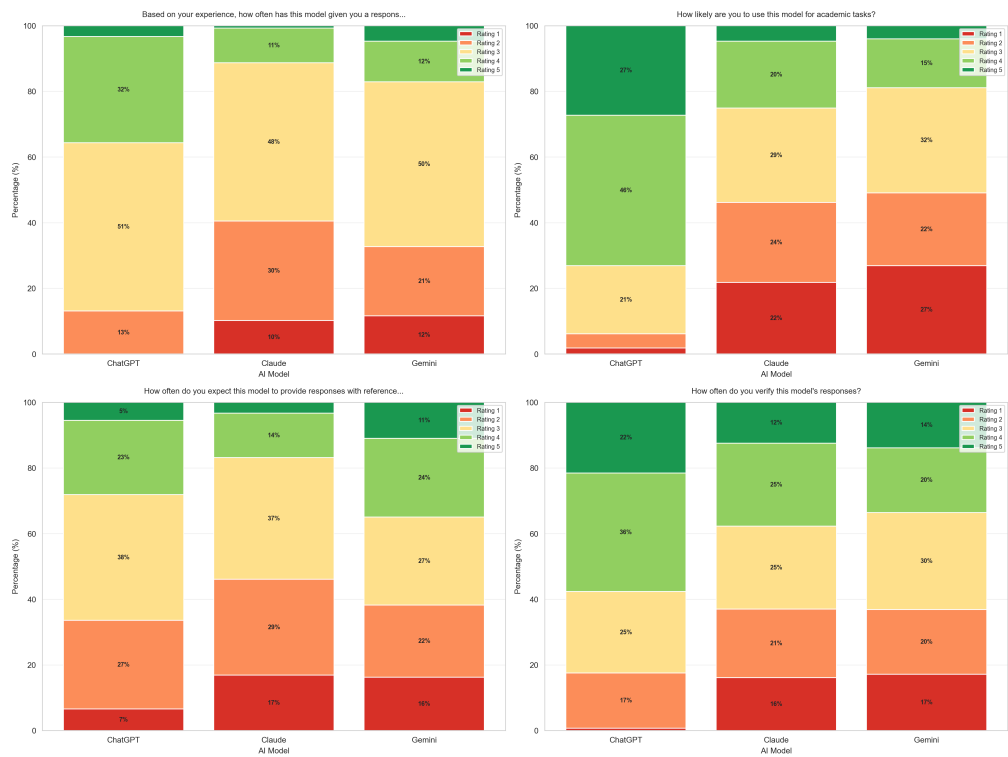
References

- [1] Claude Sammut and Geoffrey I. Webb, editors. *TF-IDF*, pages 986–987. Springer US, Boston, MA, 2010.

7 Appendix



(a) Boxplot of ratings



(b) Distribution of ratings for each model

Figure 1: Data Exploration — Comparative analysis of model ratings

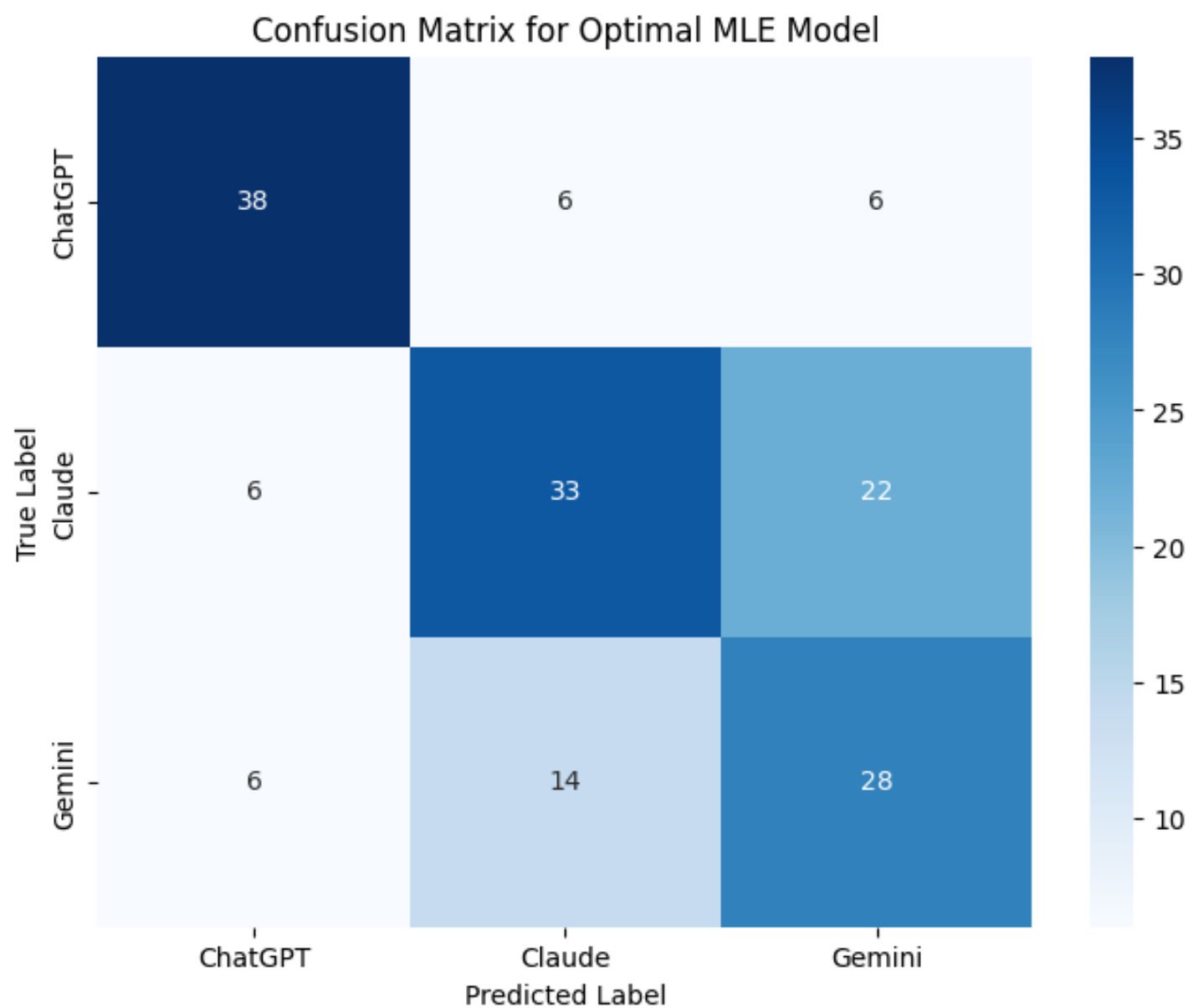


Figure 2: Confusion Matrix Validation

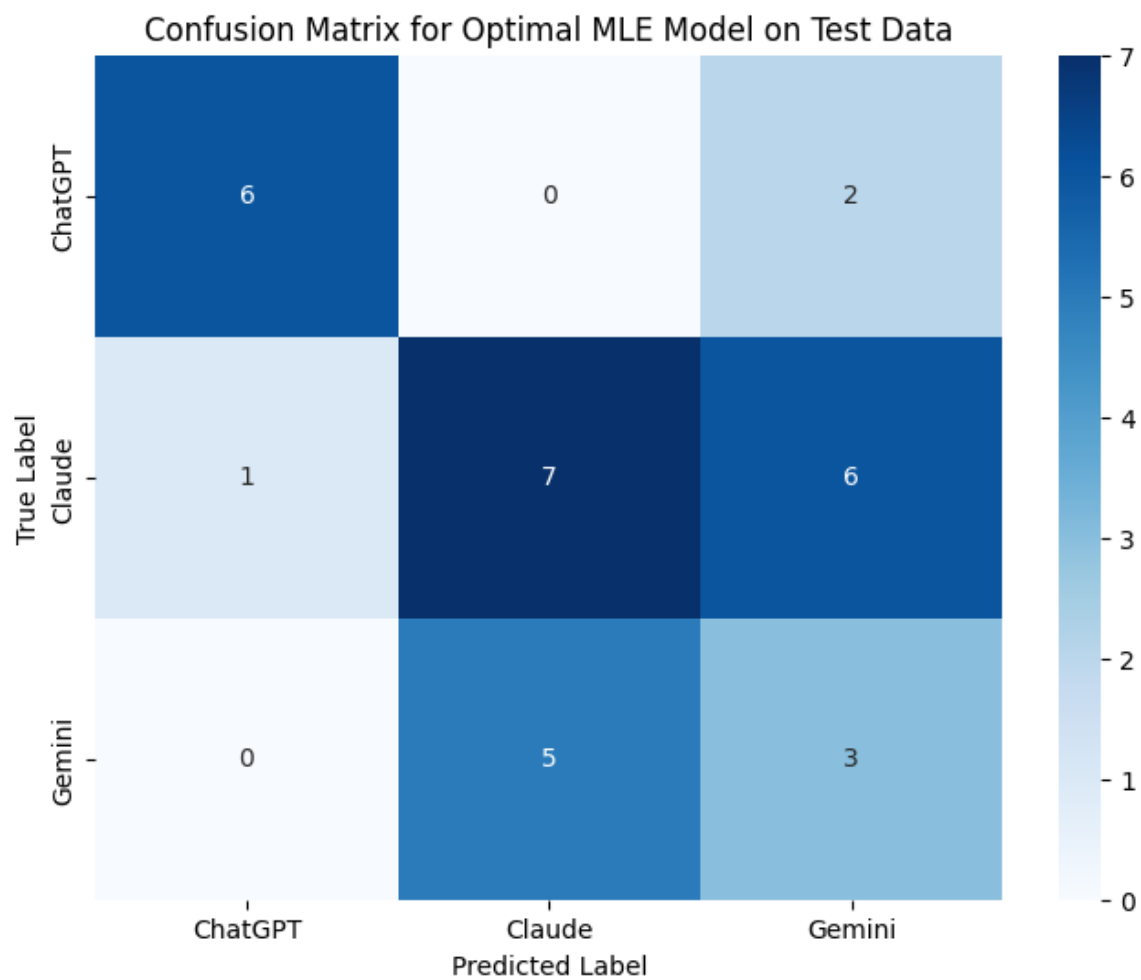


Figure 3: Confusion Matrix Test

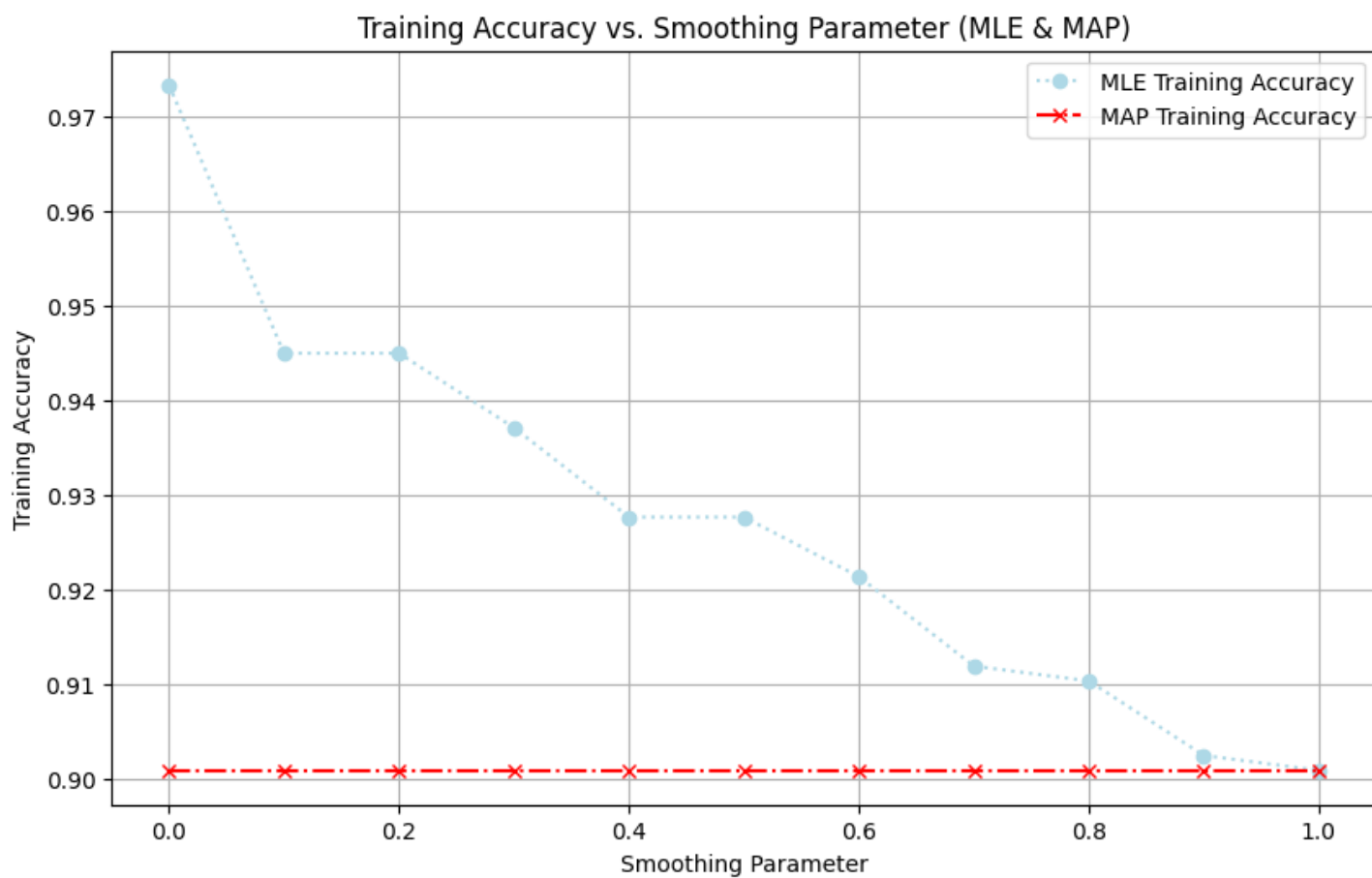


Figure 4: Naive Bayes Training vs. Smoothing

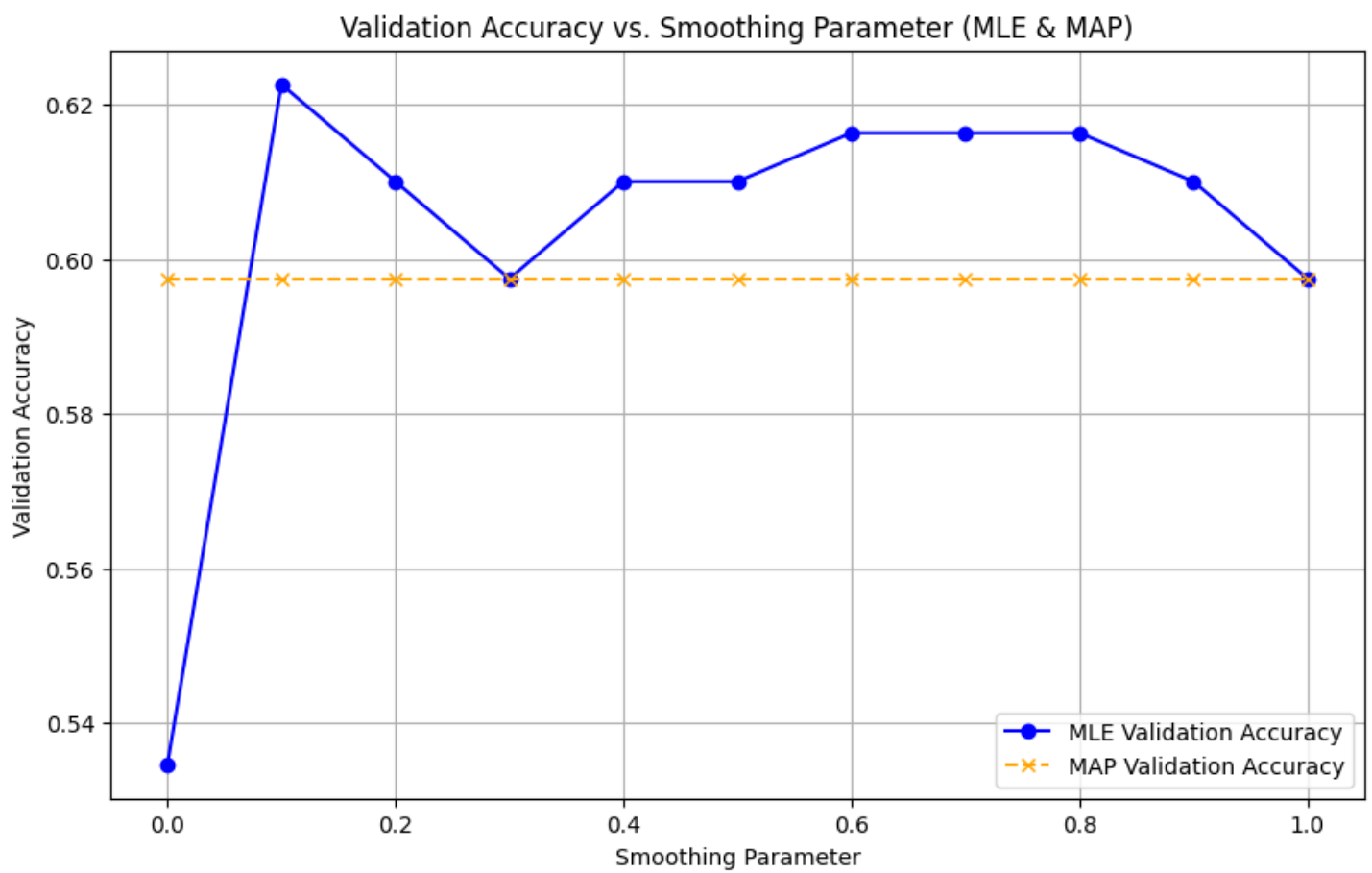


Figure 5: Naive Bayes Valid vs. Smoothing

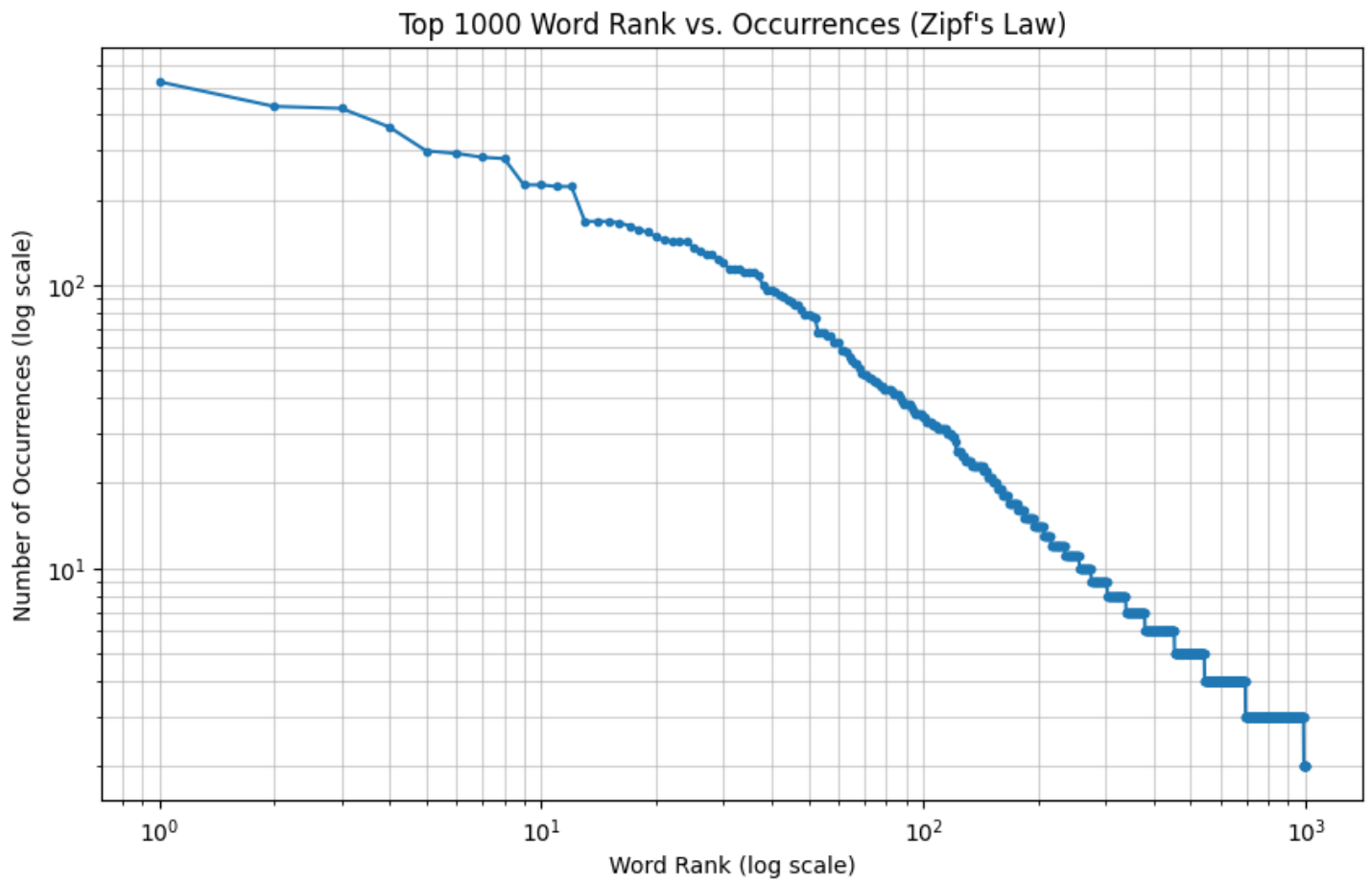
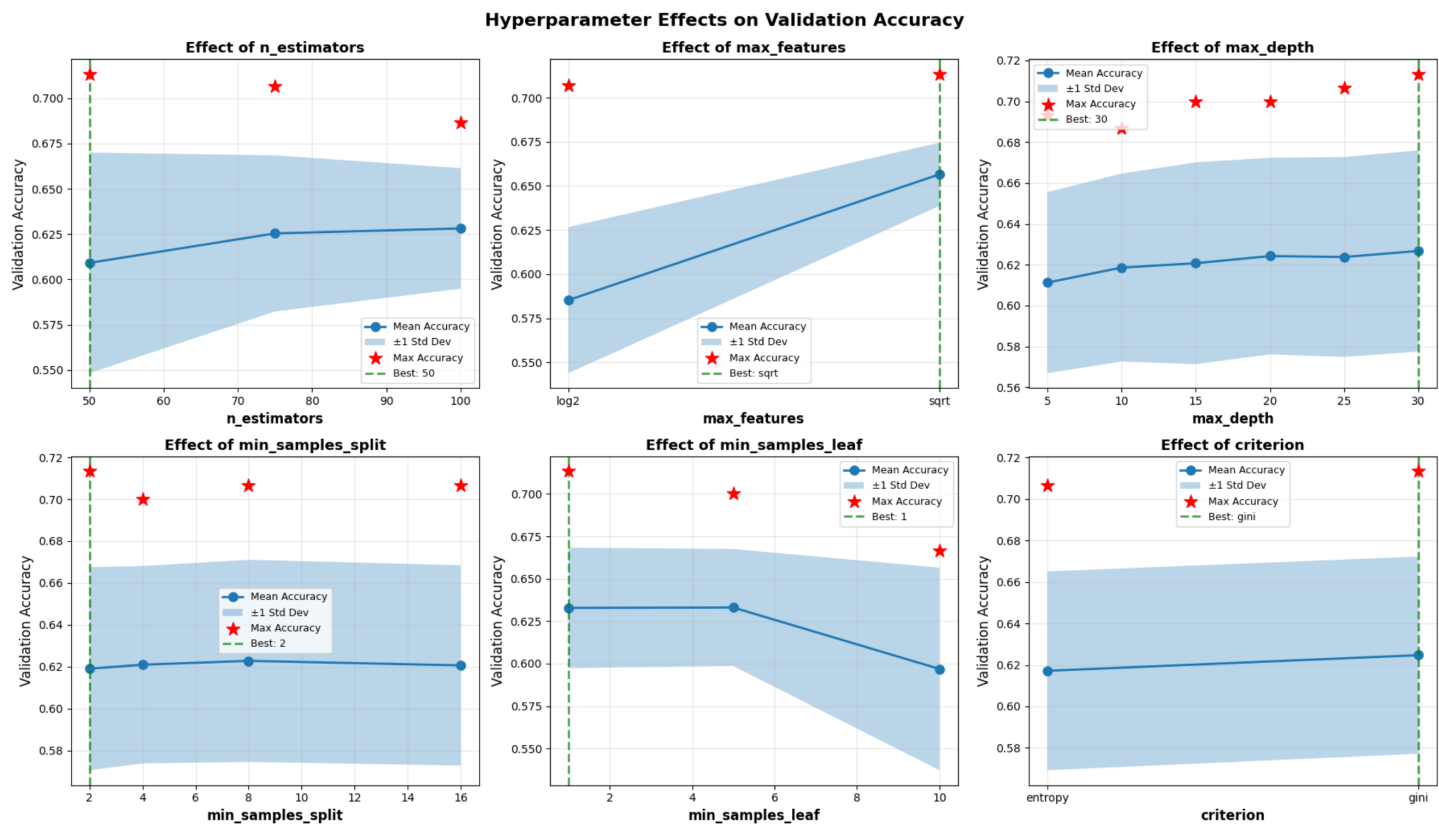
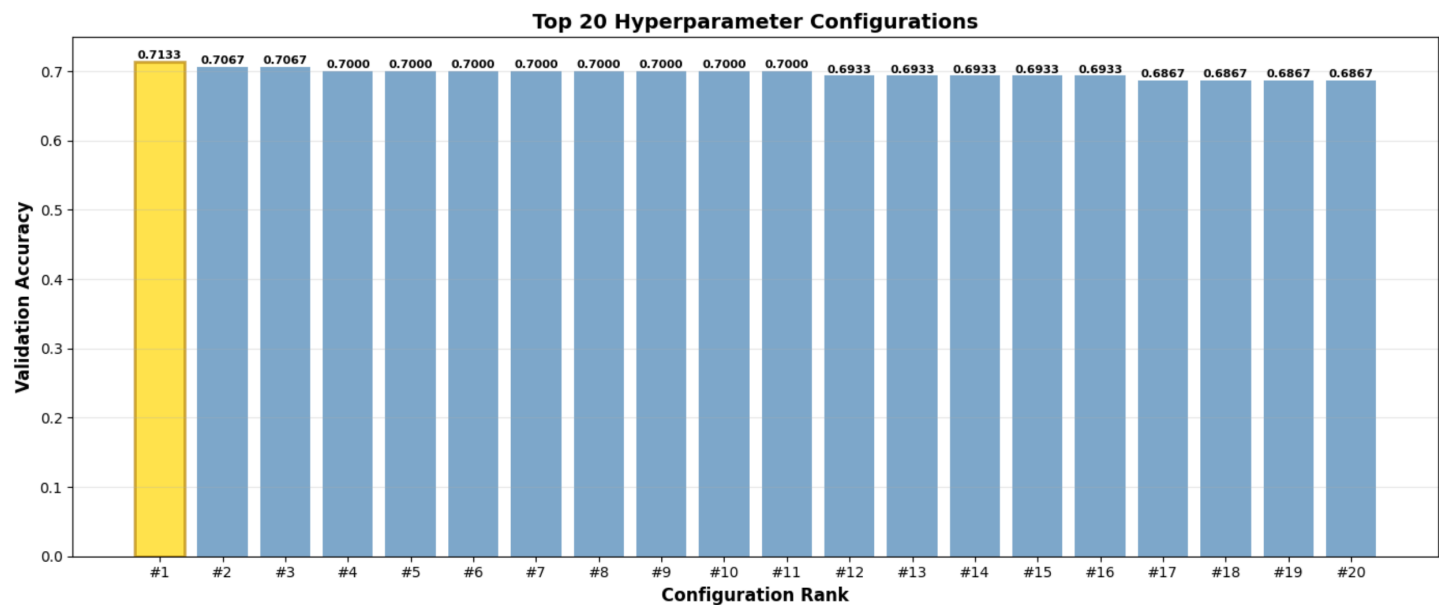


Figure 6: Word Rank



(a) RF Hyperparameter Analysis



(b) RF Hyperparamter Configurations

Figure 7: Random Forest — Hyperparamter Analysis and Configuration

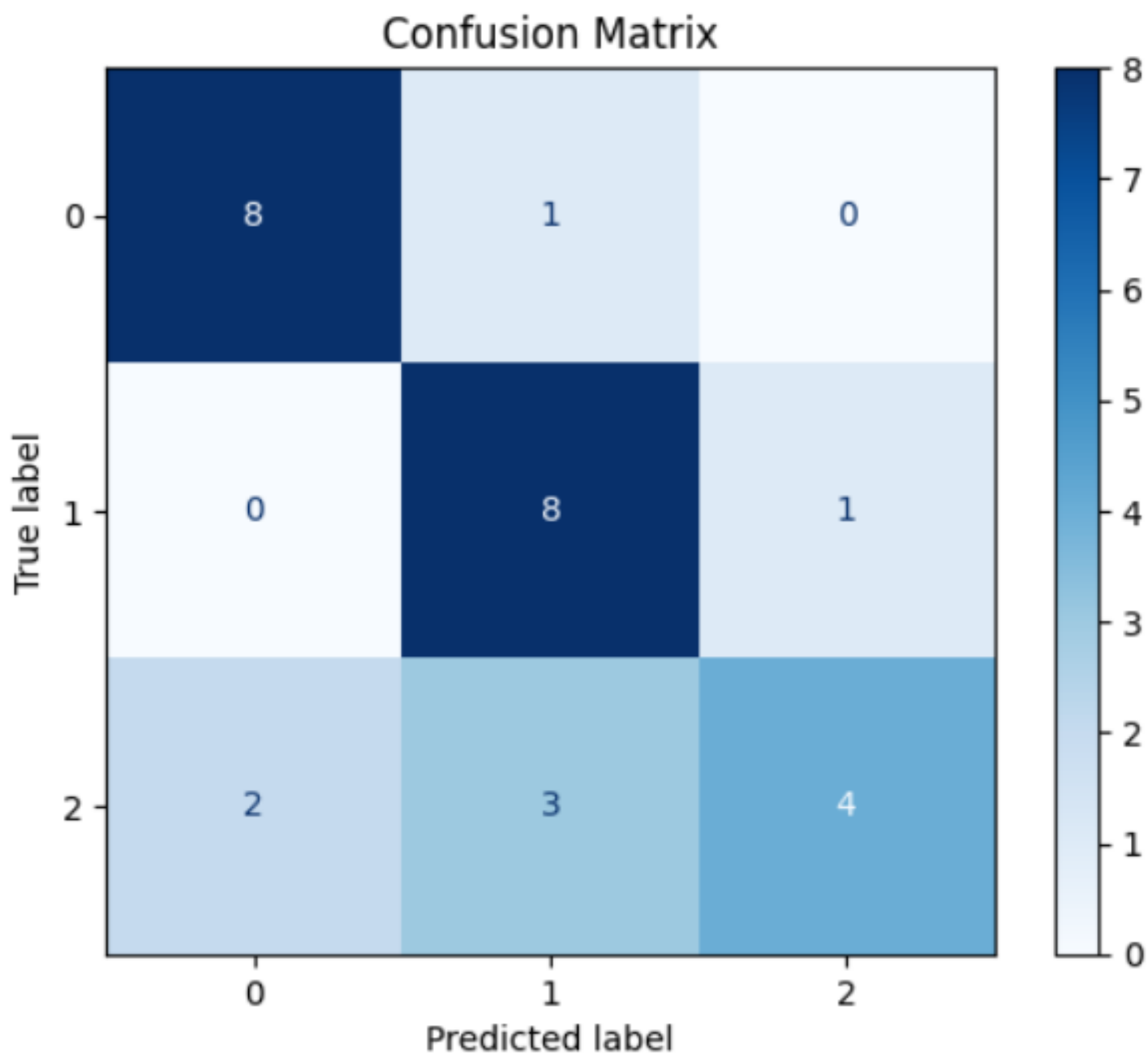


Figure 8: Random Forest — Confusion Matrix

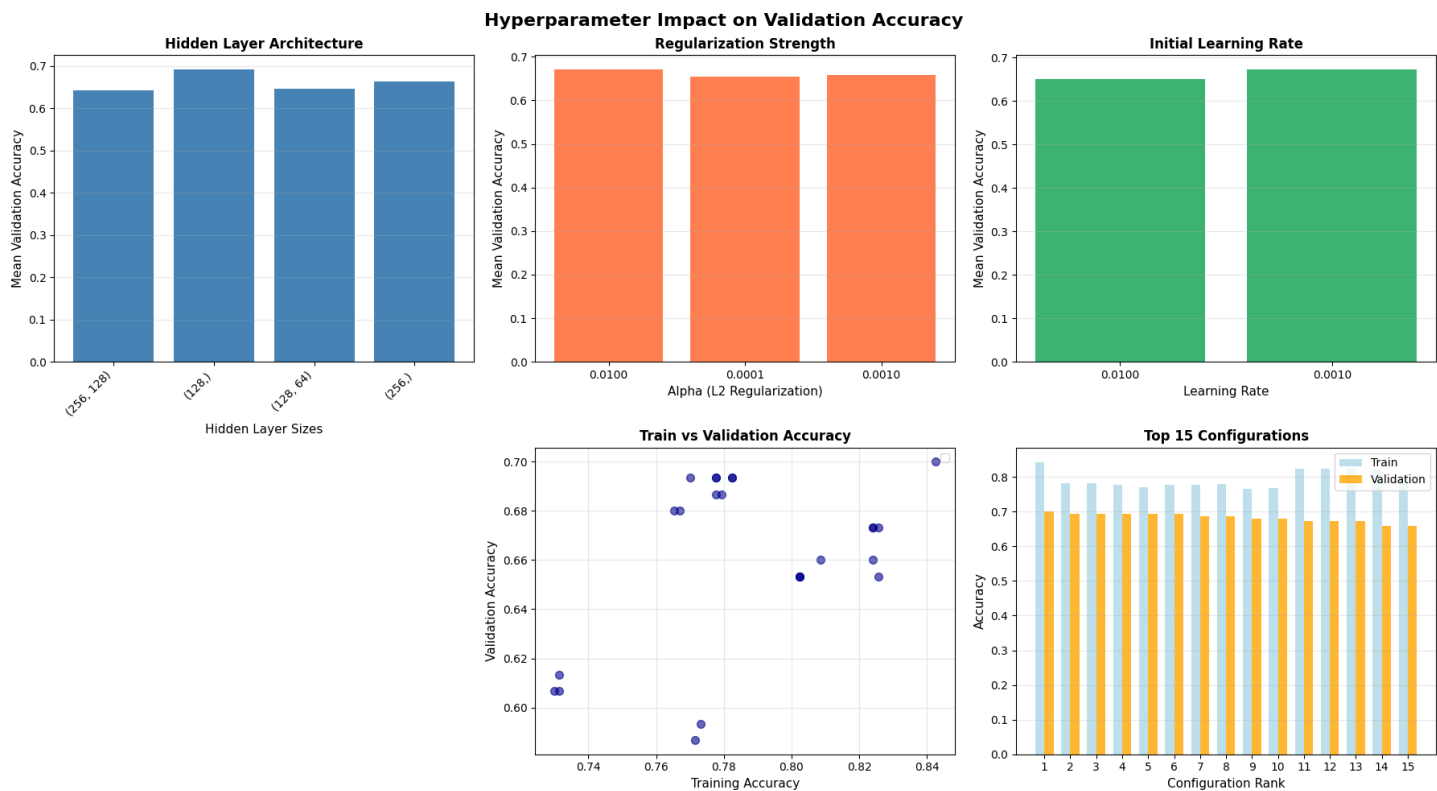


Figure 9: Hyperparameter tuning accuracies

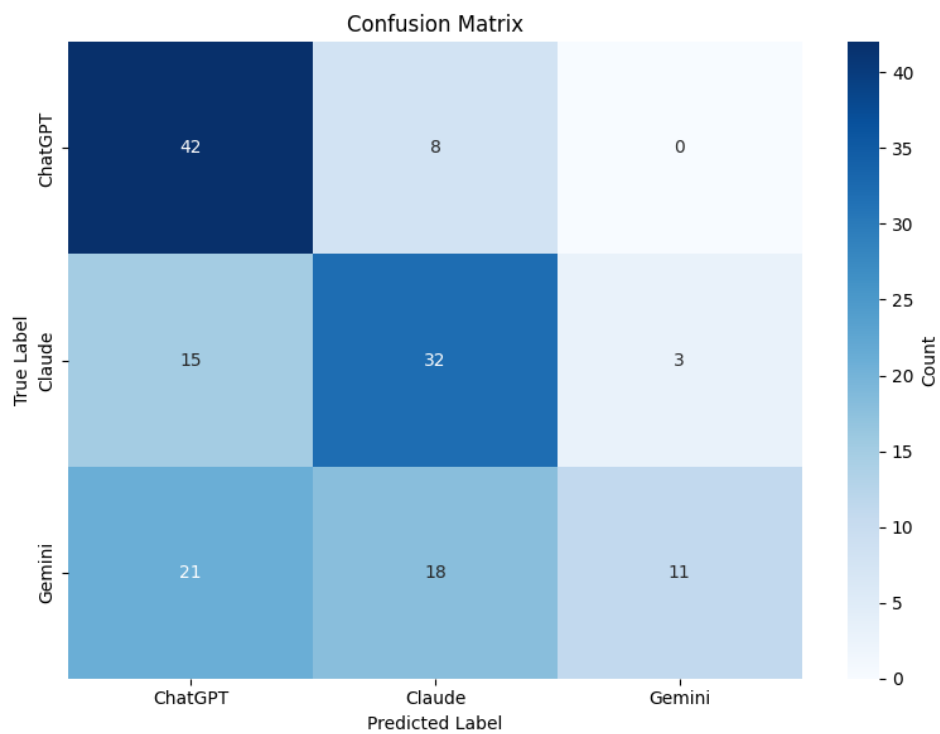


Figure 10: Confusion Matrix for MLPClassifier on Validation Set